DATA 260 Homework 8
Kafka Request Response Services Multi Agent Pipelines and Failure Recovery

Student: Sanjana Thummalapalli
SID4: 7801
Domain: Clinical Trials
GitHub Repository: https://github.com/sanjana-glitch-art/data260-7801
Submission Branch: hw8

Personal Configuration

Value	Result
SID4	7801
PORT_BASE	8601
PREFIX	s7801
SEED	7801
VERIFY_SEED	267801
DOMAIN_ID	1
Assigned Domain	Clinical Trials
Operating System	Windows
Kafka	Apache Kafka 3.9.1
JMeter	Apache JMeter 5.6.3
Java	Eclipse Temurin OpenJDK 17.0.21
Local model	No LLM was required. The Kafka agents use deterministic Python logic.

Overview

This homework extends the existing clinical-trial application with Kafka-based communication. Part 1 implements a request/response service in which FastAPI and a Kafka worker communicate through request and response topics. Part 2 implements Planner, Writer, and Reviewer as independent processes that communicate only through Kafka. Part 3 evaluates duplicate delivery, consumer termination, poison messages, dead-letter handling, partitioning, and ordering.

The completed system preserved correlation IDs across every stage, successfully handled CREATE and GET operations, recovered unfinished work after Writer termination, prevented duplicate side effects, isolated poison messages, and preserved related-message order using an entity-based Kafka key.

Part 1 Backend Service Request Response over Kafka

Architecture

The request/response service uses exactly two prefixed Kafka topics:

Purpose	Topic
Requests from client or FastAPI	s7801_entity_requests
Responses from worker	s7801_entity_responses

The client creates one correlation ID before publishing a request. The worker consumes the request, validates it, executes the requested operation, and publishes a response containing the unchanged correlation ID. The client consumes responses until it finds the matching correlation ID.

Request Schema

```
{
 "correlation_id": "unique-uuid",
 "reply_to": "s7801_entity_responses",
 "operation": "CREATE_CLINICAL_TRIAL",
 "data": {
  "trial_title": "Kafka Clinical Trial Coordination Study",
  "trial_description": "Clinical trial description",
  "trial_phase": "Phase II",
  "sponsor_name": "San Jose State University",
  "enrollment_target": 150
 }
}
```

Response Schema

A successful CREATE operation returns:

```
{
 "correlation_id": "same-uuid",
 "success": true,
 "entity_id": "generated-uuid",
 "message": "Clinical trial created"
}
```

A successful GET operation returns:

```
{
 "correlation_id": "same-uuid",
 "success": true,
 "entity": {
  "entity_id": "generated-uuid",
  "trial_title": "Kafka Clinical Trial Coordination Study",
  "trial_phase": "Phase II",
  "sponsor_name": "San Jose State University",
  "enrollment_target": 150
 }
}
```

Kafka Client

The client records the response topic's current high watermark before sending a request. It then waits only for responses produced after the request began and returns the response with the matching correlation ID.

response_partition = TopicPartition(response_topic, 0,)_, high_watermark = consumer.get_watermark_offsets(response_partition, cached=False,)consumer.assign([TopicPartition(response_topic, 0, high_watermark,)])

```
Student: Sanjana Thummalapalli | SID4: 7801 | DATA-260 HW8

  code\hw8_kafka_client.py:116:          response_partition = (
  code\hw8_kafka_client.py:117:              TopicPartition(
  code\hw8_kafka_client.py:118:                  self.response_topic,
  code\hw8_kafka_client.py:119:                  0,
  code\hw8_kafka_client.py:120:              )
  code\hw8_kafka_client.py:121:          )
  code\hw8_kafka_client.py:122:
  code\hw8_kafka_client.py:123:          _low_offset, high_offset = (
> code\hw8_kafka_client.py:124:              consumer.get_watermark_offsets(
  code\hw8_kafka_client.py:125:                  response_partition,
  code\hw8_kafka_client.py:126:                  timeout=5.0,
  code\hw8_kafka_client.py:127:                  cached=False,
  code\hw8_kafka_client.py:128:              )
  code\hw8_kafka_client.py:129:          )
  code\hw8_kafka_client.py:130:
> code\hw8_kafka_client.py:131:          consumer.assign([
  code\hw8_kafka_client.py:132:              TopicPartition(
  code\hw8_kafka_client.py:133:                  self.response_topic,
  code\hw8_kafka_client.py:134:                  0,
  code\hw8_kafka_client.py:135:                  high_offset,
  code\hw8_kafka_client.py:136:              )
  code\hw8_kafka_client.py:137:          ])
  code\hw8_kafka_client.py:138:
  code\hw8_kafka_client.py:139:          print(
> code\hw8_kafka_client.py:140:              "CLIENT_SEND | "
  code\hw8_kafka_client.py:141:              + json.dumps(
  code\hw8_kafka_client.py:142:                  request_message,
  code\hw8_kafka_client.py:143:                  ensure_ascii=False,
  code\hw8_kafka_client.py:144:              ),
  code\hw8_kafka_client.py:145:              flush=True,
  code\hw8_kafka_client.py:146:          )
  code\hw8_kafka_client.py:147:
  code\hw8_kafka_client.py:148:          producer.produce(
  code\hw8_kafka_client.py:149:              self.request_topic,
  code\hw8_kafka_client.py:150:              key=request_correlation_id,
  code\hw8_kafka_client.py:151:              value=json_bytes(
  code\hw8_kafka_client.py:152:                  request_message
  code\hw8_kafka_client.py:186:                  response.get(
  code\hw8_kafka_client.py:187:                      "correlation_id"
  code\hw8_kafka_client.py:188:                  )
  code\hw8_kafka_client.py:189:                  != request_correlation_id
  code\hw8_kafka_client.py:190:              ):
  code\hw8_kafka_client.py:191:                  continue
  code\hw8_kafka_client.py:192:
  code\hw8_kafka_client.py:193:              print(
> code\hw8_kafka_client.py:194:                  "CLIENT_RECEIVE | "
  code\hw8_kafka_client.py:195:                  + json.dumps(
  code\hw8_kafka_client.py:196:                      response,
  code\hw8_kafka_client.py:197:                      ensure_ascii=False,
  code\hw8_kafka_client.py:198:                  ),
  code\hw8_kafka_client.py:199:                  flush=True,
  code\hw8_kafka_client.py:200:              )
  code\hw8_kafka_client.py:201:
  code\hw8_kafka_client.py:202:              return response
  code\hw8_kafka_client.py:203:
  code\hw8_kafka_client.py:204:          raise KafkaRequestError(
  code\hw8_kafka_client.py:205:              "Timed out waiting for the "
  code\hw8_kafka_client.py:206:              "correlated Kafka response."
```

Figure 1: Kafka client code from code/hw8_kafka_client.py

Worker

The worker stores clinical-trial entities in a Python dictionary and generates entity IDs using UUID.

entity_id = str(uuid4())

entity = {

```
    "entity_id": entity_id,
    **validated_data,
}

ENTITY_STORE[entity_id] = entity
```

The worker validates:

- Required fields
- Non-empty string fields
- Valid trial phase
- Positive enrollment target
- Known operation names
- Existing entity IDs for GET requests

Unknown operations and missing entities return clean error messages with the original correlation ID.

```
Student: Sanjana Thummalapalli | SID4: 7801 | DATA-260 HW8

  code\hw8_worker.py:183:               validation_error,
  code\hw8_worker.py:184:           )
  code\hw8_worker.py:185:
  code\hw8_worker.py:186:       assert trial_data is not None
  code\hw8_worker.py:187:
> code\hw8_worker.py:188:       entity_id = str(uuid4())
  code\hw8_worker.py:189:
  code\hw8_worker.py:190:       stored_trial = {
  code\hw8_worker.py:191:           "entity_id": entity_id,
  code\hw8_worker.py:192:           **trial_data,
  code\hw8_worker.py:193:       }
  code\hw8_worker.py:194:
  code\hw8_worker.py:195:       TRIAL_STORE[
  code\hw8_worker.py:196:           entity_id
  code\hw8_worker.py:197:       ] = stored_trial
  code\hw8_worker.py:198:
  code\hw8_worker.py:199:       response = {
  code\hw8_worker.py:200:           "correlation_id": (
  code\hw8_worker.py:201:               correlation_id
  code\hw8_worker.py:202:           ),
  code\hw8_worker.py:203:           "success": True,
  code\hw8_worker.py:204:           "entity_id": entity_id,
  code\hw8_worker.py:205:           "message": (
  code\hw8_worker.py:206:               "Clinical trial created"
```

Figure 2: Worker operation code from code/hw8_worker.py

FastAPI Wrapper

FastAPI publishes the request to Kafka and waits for the correlated response.

```
response = client.request(    operation=request.operation,    data=request.data,    correlation_id=request_correlation_id,)
```

The API endpoints are:

```
POST /hw8/trials
GET  /hw8/trials/{entity_id}
GET  /health
```



```
Student: Sanjana Thummalapalli | SID4: 7801 | DATA-260 HW8

  code\hw8_api.py:148:
  code\hw8_api.py:149:
> code\hw8_api.py:150:@app.get("/")
  code\hw8_api.py:151:def read_root() -> dict[str, Any]:
  code\hw8_api.py:152:    """Return Homework 8 service information."""
  code\hw8_api.py:153:
  code\hw8_api.py:154:    return {
  code\hw8_api.py:155:        "application": (
  code\hw8_api.py:156:            "DATA-260 Homework 8 Kafka API"
  code\hw8_api.py:157:        ),
  code\hw8_api.py:158:        "student": (
  code\hw8_api.py:159:            "Sanjana Thummalapalli"
  code\hw8_api.py:160:        ),
  code\hw8_api.py:161:        "sid4": 7801,
  code\hw8_api.py:162:        "prefix": PREFIX,
  code\hw8_api.py:163:        "port": PORT_BASE,
  code\hw8_api.py:164:        "request_topic": REQUEST_TOPIC,
  code\hw8_api.py:165:        "response_topic": RESPONSE_TOPIC,
  code\hw8_api.py:166:    }
  code\hw8_api.py:167:
  code\hw8_api.py:168:
> code\hw8_api.py:169:@app.get("/health")
  code\hw8_api.py:170:def health_check() -> dict[str, Any]:
  code\hw8_api.py:171:    """Return the API process health state."""
  code\hw8_api.py:172:
  code\hw8_api.py:173:    return {
  code\hw8_api.py:174:        "status": "ok",
  code\hw8_api.py:175:        "kafka_bootstrap_servers": (
  code\hw8_api.py:176:            BOOTSTRAP_SERVERS
  code\hw8_api.py:177:        ),
  code\hw8_api.py:178:        "request_topic": REQUEST_TOPIC,
  code\hw8_api.py:179:        "response_topic": RESPONSE_TOPIC,
  code\hw8_api.py:180:    }
  code\hw8_api.py:181:
  code\hw8_api.py:182:
> code\hw8_api.py:183:@app.post(
  code\hw8_api.py:184:    "/hw8/trials",
  code\hw8_api.py:185:    status_code=201,
  code\hw8_api.py:186:)
  code\hw8_api.py:187:def create_trial(
  code\hw8_api.py:188:    request: CreateTrialRequest,
  code\hw8_api.py:189:) -> dict[str, Any]:
  code\hw8_api.py:190:    """Create a clinical trial through Kafka."""
  code\hw8_api.py:191:
  code\hw8_api.py:192:    correlation_id = (
  code\hw8_api.py:193:        request.correlation_id
  code\hw8_api.py:194:        or str(uuid4())
  code\hw8_api.py:195:    )
  code\hw8_api.py:196:
  code\hw8_api.py:197:    return send_request(
  code\hw8_api.py:198:        CREATE_OPERATION,
  code\hw8_api.py:199:        request.data.model_dump(),
  code\hw8_api.py:200:        correlation_id,
  code\hw8_api.py:201:    )
  code\hw8_api.py:202:
  code\hw8_api.py:203:
> code\hw8_api.py:204:@app.get("/hw8/trials/{entity_id}")
  code\hw8_api.py:205:def get_trial(
  code\hw8_api.py:206:    entity_id: str,
  code\hw8_api.py:207:    correlation_id: str | None = Query(
  code\hw8_api.py:208:        default=None,
  code\hw8_api.py:209:    ),
  code\hw8_api.py:210:) -> dict[str, Any]:
  code\hw8_api.py:211:    """Retrieve one clinical trial through Kafka."""
  code\hw8_api.py:212:
```

Figure 3: FastAPI wrapper from code/hw8_api.py

Complete CREATE and GET Demonstration

The demonstration performed these steps:

1. Created one correlation ID.
2. Published CREATE_CLINICAL_TRIAL.
3. Logged the request sent to Kafka.
4. Logged worker receipt and processing.
5. Returned the generated UUID.
6. Published GET_CLINICAL_TRIAL using that UUID.
7. Retrieved the stored clinical trial.
8. Confirmed correlation-ID preservation.

Representative result:

CREATE -> GET round trip: PASS
Correlation ID preserved: PASS

The retrieved record contained the generated entity ID, title, description, phase, sponsor, and enrollment target.

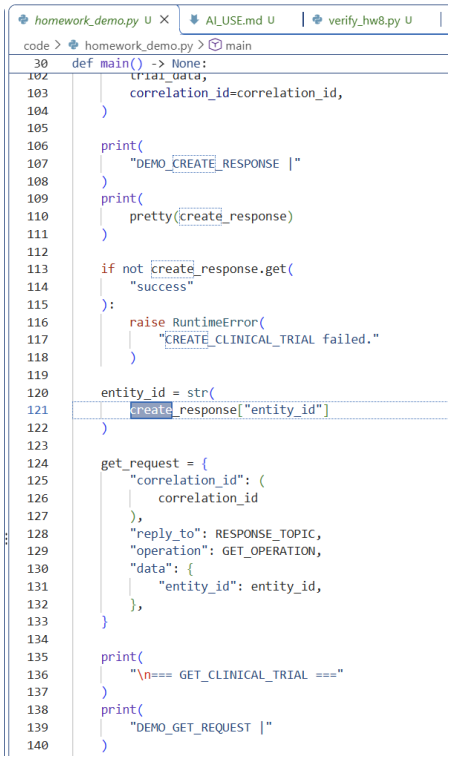


Figure 4 : homework_demo.py CREATE response

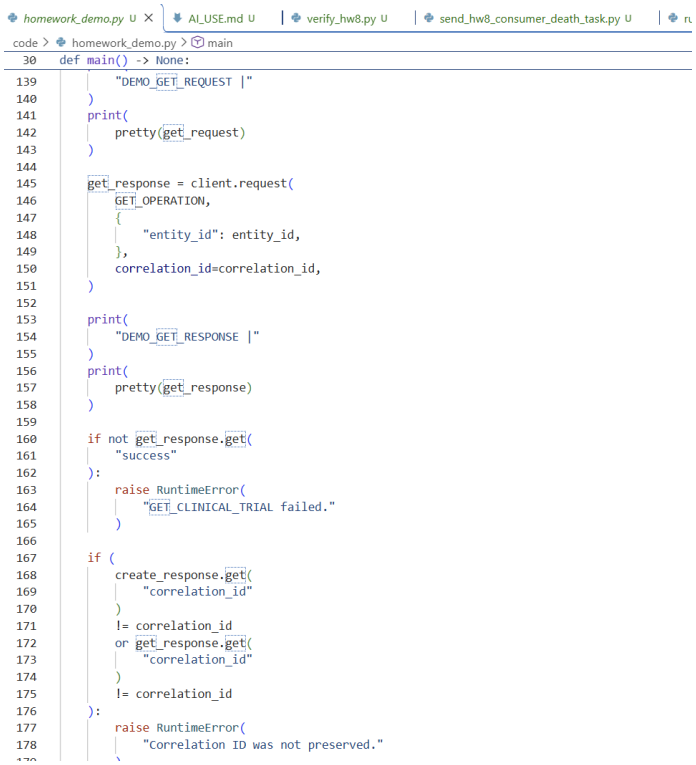


Figure 5: homework_demo.py GET response and final PASS confirmation

```
Student: Sanjana Thummalapalli | SID4: 7801 | DATA-260 HW8

=== WORKER LOGS ===
WORKER_RECEIVE | {"correlation_id": "01f8a5c1-6548-499d-bb1c-1c306d29fcb0", "reply_to": "s7801_entity_responses", "operation": "CREATE_CLINICAL_TRIAL", "data": {"trial_title": "JMeter Kafka Clinical Trial 100", "trial_descript
ion": "This clinical trial was created during the DATA-260 Homework 8 JMeter load test.", "trial_phase": "Phase II", "sponsor_name": "San Jose State University", "enrollment_target": 150}}
WORKER_PROCESS | CREATE_CLINICAL_TRIAL
WORKER_REPLY | {"correlation_id": "01f8a5c1-6548-499d-bb1c-1c306d29fcb0", "success": true, "entity_id": "b2e15449-8268-4847-8bc9-715a26bb564b", "message": "Clinical trial created", "idempotent_replay": false}
WORKER_COMMIT | 01f8a5c1-6548-499d-bb1c-1c306d29fcb0
WORKER_RECEIVE | {"correlation_id": "dc727c96-b593-4650-b77b-5025798e48e5", "reply_to": "s7801_entity_responses", "operation": "CREATE_CLINICAL_TRIAL", "data": {"trial_title": "Kafka Clinical Trial Coordination Study", "trial_
description": "This Homework 8 clinical trial demonstrates Kafka request and response message coordination.", "trial_phase": "Phase II", "sponsor_name": "San Jose State University", "enrollment_target": 150}}
WORKER_PROCESS | CREATE_CLINICAL_TRIAL
WORKER_REPLY | {"correlation_id": "dc727c96-b593-4650-b77b-5025798e48e5", "success": true, "entity_id": "8a0cc725-8f92-4fe6-91df-6795560c0ef5", "message": "Clinical trial created", "idempotent_replay": false}
WORKER_COMMIT | dc727c96-b593-4650-b77b-5025798e48e5
WORKER_RECEIVE | {"correlation_id": "dc727c96-b593-4650-b77b-5025798e48e5", "reply_to": "s7801_entity_responses", "operation": "GET_CLINICAL_TRIAL", "data": {"entity_id": "8a0cc725-8f92-4fe6-91df-6795560c0ef5"}}
WORKER_PROCESS | GET_CLINICAL_TRIAL
WORKER_REPLY | {"correlation_id": "dc727c96-b593-4650-b77b-5025798e48e5", "success": true, "entity": {"entity_id": "8a0cc725-8f92-4fe6-91df-6795560c0ef5", "trial_title": "Kafka Clinical Trial Coordination Study", "trial_descri
ption": "This Homework 8 clinical trial demonstrates Kafka request and response message coordination.", "trial_phase": "Phase II", "sponsor_name": "San Jose State University", "enrollment_target": 150}}
WORKER_COMMIT | dc727c96-b593-4650-b77b-5025798e48e5

=== FASTAPI LOGS ===
API_RESPONSE | {"correlation_id": "9465132c-fa1d-4603-b700-37769c0729bc", "success": true, "entity_id": "78ffa500-2933-45cf-9b32-718a3e56272f", "message": "Clinical trial created", "idempotent_replay": false}
INFO:     172.19.0.1:45930 - "POST /hw8/trials HTTP/1.1" 201 Created
API_REQUEST | {"correlation_id": "3b9005f3-3338-497b-aad9-854fc9a787dd", "operation": "CREATE_CLINICAL_TRIAL", "data": {"trial_title": "JMeter Kafka Clinical Trial 99", "trial_description": "This clinical trial was created dur
ing the DATA-260 Homework 8 JMeter load test.", "trial_phase": "Phase II", "sponsor_name": "San Jose State University", "enrollment_target": 150}}
CLIENT_SEND | {"correlation_id": "3b9005f3-3338-497b-aad9-854fc9a787dd", "reply_to": "s7801_entity_responses", "operation": "CREATE_CLINICAL_TRIAL", "data": {"trial_title": "JMeter Kafka Clinical Trial 99", "trial_description"
: "This clinical trial was created during the DATA-260 Homework 8 JMeter load test.", "trial_phase": "Phase II", "sponsor_name": "San Jose State University", "enrollment_target": 150}}
CLIENT_RECEIVE | {"correlation_id": "3b9005f3-3338-497b-aad9-854fc9a787dd", "success": true, "entity_id": "42699b95-bbde-4ab4-a3ae-732371f7d68e", "message": "Clinical trial created", "idempotent_replay": false}
API_RESPONSE | {"correlation_id": "3b9005f3-3338-497b-aad9-854fc9a787dd", "success": true, "entity_id": "42699b95-bbde-4ab4-a3ae-732371f7d68e", "message": "Clinical trial created", "idempotent_replay": false}
INFO:     172.19.0.1:45946 - "POST /hw8/trials HTTP/1.1" 201 Created
API_REQUEST | {"correlation_id": "01f8a5c1-6548-499d-bb1c-1c306d29fcb0", "operation": "CREATE_CLINICAL_TRIAL", "data": {"trial_title": "JMeter Kafka Clinical Trial 100", "trial_description": "This clinical trial was created du
ring the DATA-260 Homework 8 JMeter load test.", "trial_phase": "Phase II", "sponsor_name": "San Jose State University", "enrollment_target": 150}}
CLIENT_SEND | {"correlation_id": "01f8a5c1-6548-499d-bb1c-1c306d29fcb0", "reply_to": "s7801_entity_responses", "operation": "CREATE_CLINICAL_TRIAL", "data": {"trial_title": "JMeter Kafka Clinical Trial 100", "trial_description
": "This clinical trial was created during the DATA-260 Homework 8 JMeter load test.", "trial_phase": "Phase II", "sponsor_name": "San Jose State University", "enrollment_target": 150}}
CLIENT_RECEIVE | {"correlation_id": "01f8a5c1-6548-499d-bb1c-1c306d29fcb0", "success": true, "entity_id": "b2e15449-8268-4847-8bc9-715a26bb564b", "message": "Clinical trial created", "idempotent_replay": false}
API_RESPONSE | {"correlation_id": "01f8a5c1-6548-499d-bb1c-1c306d29fcb0", "success": true, "entity_id": "b2e15449-8268-4847-8bc9-715a26bb564b", "message": "Clinical trial created", "idempotent_replay": false}
INFO:     172.19.0.1:45948 - "POST /hw8/trials HTTP/1.1" 201 Created
```

Figure 6: code/homework_demo.py source code

Docker Runtime

The Docker Compose environment ran:

- s7801-kafka
- s7801-hw8-worker
- s7801-hw8-api
- s7801-kafka-init

Kafka was healthy, the worker displayed Consumer is ready and listening..., and FastAPI listened on port 8601.

```
Student: Sanjana Thummalapalli | SID4: 7801 | DATA-260 HW8

=== Docker Containers Running ===
NAME              IMAGE                    COMMAND                  SERVICE      CREATED         STATUS                     PORTS
s7801-hw8-api     s7801-hw8-hw8-api        "python -m uvicorn c…"   hw8-api      25 hours ago    Up 25 hours                0.0.0.0:8601->8601/tcp, [::]:8601->8601/tcp
s7801-hw8-worker  s7801-hw8-hw8-worker     "python -m code.hw8_…"   hw8-worker   25 hours ago    Up 25 hours                8601/tcp
s7801-kafka       apache/kafka:3.9.1       "/__cacert_entrypoin…"   kafka        25 hours ago    Up 25 hours (healthy)      0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp
s7801-kafka-init  apache/kafka:3.9.1       "/__cacert_entrypoin…"   kafka-init   25 hours ago    Exited (0) 25 hours ago
```

Figure 7: Docker containers running

```
Student: Sanjana Thummalapalli | SID4: 7801 | DATA-260 HW8

=== Kafka Worker Readiness ===

WORKER_RECEIVE | {"correlation_id": "01f8a5c1-6548-499d-bb1c-1c306d29fcb0", "reply_to": "s7801_entity_responses", "operation": "CREATE_CLINICAL_TRIAL", "data": {"trial_title": "JMeter Kafka Clinical Trial 100",
"trial_description": "This clinical trial was created during the DATA-260 Homework 8 JMeter load test.", "trial_phase": "Phase II", "sponsor_name": "San Jose State University", "enrollment_target": 150}}
WORKER_PROCESS | CREATE_CLINICAL_TRIAL
WORKER_REPLY | {"correlation_id": "01f8a5c1-6548-499d-bb1c-1c306d29fcb0", "success": true, "entity_id": "b2e15449-8268-4847-8bc9-715a26bb564b", "message": "Clinical trial created", "idempotent_replay": false}
WORKER_COMMIT | 01f8a5c1-6548-499d-bb1c-1c306d29fcb0
WORKER_RECEIVE | {"correlation_id": "dc727c96-b593-4650-b77b-5025798e48e5", "reply_to": "s7801_entity_responses", "operation": "CREATE_CLINICAL_TRIAL", "data": {"trial_title": "Kafka Clinical Trial Coordination Study",
"trial_description": "This Homework 8 clinical trial demonstrates Kafka request and response message coordination.", "trial_phase": "Phase II", "sponsor_name": "San Jose State University", "enrollment_target": 150}}
WORKER_PROCESS | CREATE_CLINICAL_TRIAL
WORKER_REPLY | {"correlation_id": "dc727c96-b593-4650-b77b-5025798e48e5", "success": true, "entity_id": "8a0cc725-8f92-4fe6-91df-6795560c0ef5", "message": "Clinical trial created", "idempotent_replay": false}
WORKER_COMMIT | dc727c96-b593-4650-b77b-5025798e48e5
WORKER_RECEIVE | {"correlation_id": "dc727c96-b593-4650-b77b-5025798e48e5", "reply_to": "s7801_entity_responses", "operation": "GET_CLINICAL_TRIAL", "data": {"entity_id": "8a0cc725-8f92-4fe6-91df-6795560c0ef5"}}
WORKER_PROCESS | GET_CLINICAL_TRIAL
WORKER_REPLY | {"correlation_id": "dc727c96-b593-4650-b77b-5025798e48e5", "success": true, "entity": {"entity_id": "8a0cc725-8f92-4fe6-91df-6795560c0ef5", "trial_title": "Kafka Clinical Trial Coordination Study",
"trial_description": "This Homework 8 clinical trial demonstrates Kafka request and response message coordination.", "trial_phase": "Phase II", "sponsor_name": "San Jose State University", "enrollment_target": 150}}
WORKER_COMMIT | dc727c96-b593-4650-b77b-5025798e48e5

=== FastAPI Startup ===
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8601 (Press CTRL+C to quit)
```

Figure 8 : Worker readiness and FastAPI startup logs

Part 2 Three Agent Kafka Pipeline
Pipeline Design

Planner, Writer, and Reviewer run as separate Python processes. They never call each other directly.

Process	Reads	Writes
Sender	—	s7801_inbox
Planner	s7801_inbox	s7801_tasks
Writer	s7801_tasks	s7801_drafts
Reviewer	s7801_drafts	s7801_final

The s7801_tasks topic contains two partitions. The other pipeline topics contain one partition.

Planner

The Planner consumes a question and produces a structured plan.

plan_message = { "correlation_id": correlation_id, "question": question, "plan": build_plan(question), "entity_key": entity_key, "stage": "planned",}

Writer

The Writer consumes the plan and creates a draft response.

draft_message = { "correlation_id": correlation_id, "question": question, "plan": plan, "draft": build_draft(question, [str(step) for step in plan],), "entity_key": entity_key, "stage": "drafted",}

Reviewer

The Reviewer validates the draft and publishes the final approved response.

```
final_message = {
   "correlation_id": correlation_id,
   "question": question,
   "status": "approved",
   "answer": reviewed_answer,
   "entity_key": entity_key,
   "stage": "reviewed",
}
```

Pipeline Result

The verified message journey was:

```
s7801_inbox
   -> s7801_tasks
   -> s7801_drafts
   -> s7801_final
```

The final message had:

```
{
  "correlation_id": "8afd51d5-421a-4ff6-bb3b-ae5c85902d61",
  "status": "approved",
  "entity_key": "clinical-trials",
  "stage": "reviewed"
}
```


The correlation ID remained unchanged throughout all four topics.

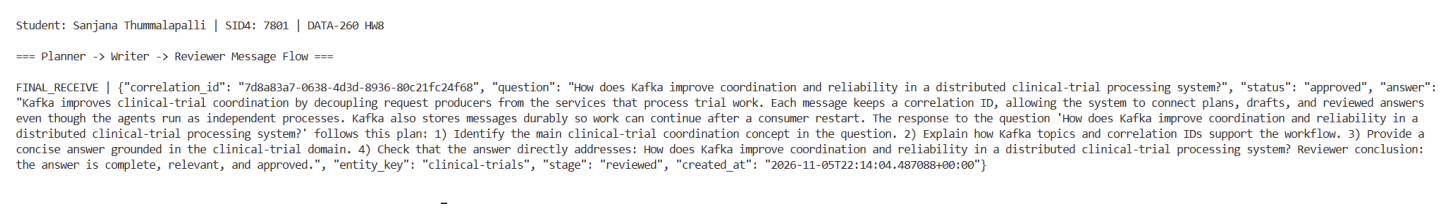


Figure 9 : Planner, Writer, and Reviewer message-flow logs

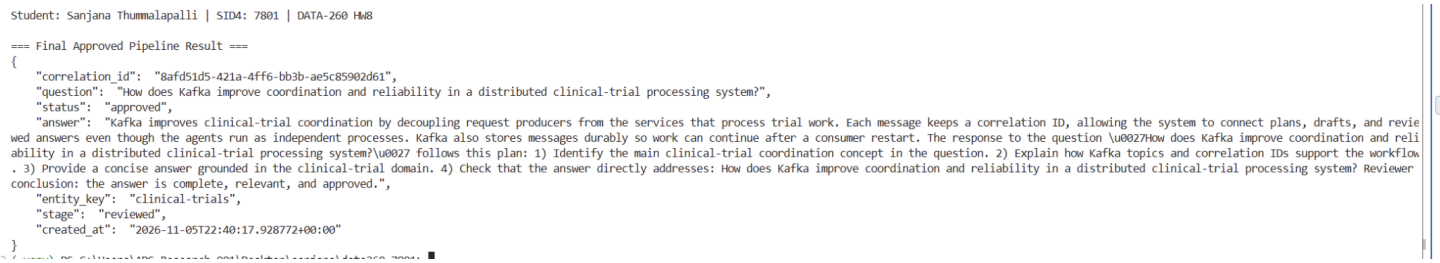


Figure 10 : Final approved JSON from pipeline_result.json

How Kafka Helped Agent Coordination

Kafka decoupled the three agents. Each process depended only on its input and output topics rather than the availability of another Python process. Kafka retained unfinished work, enabled independent process restarts, allowed multiple Writer instances to share partitions, and preserved the correlation ID across asynchronous stages.

Part 3 Failure Behavior

Duplicate Delivery

Before the Fix

The same CREATE request was replayed using correlation ID: b4406b89-cb1b-4931-94d5-2b965e602947

It generated two different entity IDs: 80f84e28-915e-41f0-89d9-cc6f59ca7daf, be035327-3b01-4bfe-8dd5-ba41971e6b36

Therefore:

Duplicate side effect: True
Idempotency passed: False

After the Fix

The worker cached CREATE responses using the correlation ID.

```
cached_response = PROCESSED_CREATE_RESPONSES.get(
   correlation_id
```

)

```
if cached_response is not None:
   return {
      **cached_response,
      "idempotent_replay": True,
   }
```

The corrected run used correlation ID: 885f1ad2-8684-495b-8588-ec50a342c9a7

Both responses returned the same entity ID: 45b7ff1c-c0f8-4c47-a2ce-000404a5f9f2

Final result:

Duplicate side effect: False
Idempotency passed: True

```
Student: Sanjana Thummalapalli | SID4: 7801 | DATA-260 HW8

=== Duplicate Delivery: Before and After Idempotency Fix ===

--- BEFORE FIX ---
Correlation ID:       b4406b89-cb1b-4931-94d5-2b965e602947
First entity ID:      80f84e28-915e-41f0-89d9-cc6f59ca7daf
Second entity ID:     be035327-3b01-4bfe-8dd5-ba41971e6b36
Duplicate side effect: True
Idempotency passed:    False

--- AFTER FIX ---
Correlation ID:       885f1ad2-8684-495b-8588-ec50a342c9a7
First entity ID:      45b7ff1c-c0f8-4c47-a2ce-000404a5f9f2
Second entity ID:     45b7ff1c-c0f8-4c47-a2ce-000404a5f9f2
Idempotent replay:    True
Duplicate side effect: False
Idempotency passed:    True
```

Figure 11: Duplicate delivery before and after results

Consumer Killed Mid Work

The Writer used manual offset commits:

consumer.commit(message=kafka_message, asynchronous=False,)

The offset was committed only after the draft was successfully published.

During the failure experiment, the Writer received the task and entered a controlled processing delay. The Writer process was forcefully terminated before WRITER_SEND and before the offset commit. After Kafka detected the failed consumer and reassigned the partition, the restarted Writer received the same correlation ID again, published the draft, and committed the offset.

WRITER_RECEIVE
WRITER_PROCESSING_DELAY
Writer terminated
WRITER_RECEIVE
WRITER_SEND
WRITER_COMMIT

This demonstrated Kafka redelivery of uncommitted work.

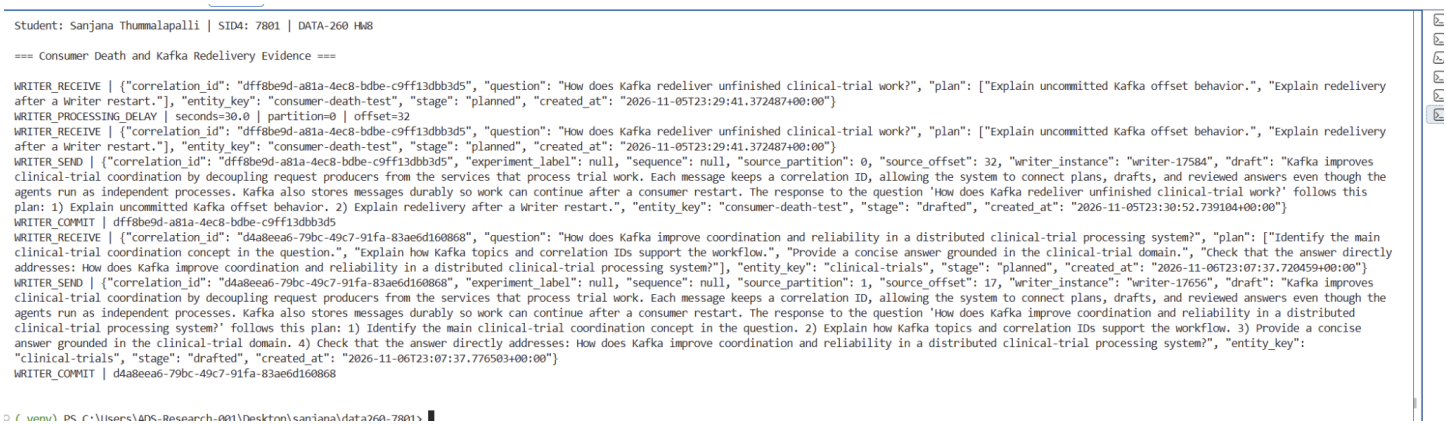


Figure 12 : Consumer death and redelivery log showing the same correlation ID twice

Poison Message and Dead Letter Topic

The Writer used a maximum of three processing attempts.

for attempt in range(1, max_processing_attempts + 1,): try: process_message() except (KeyError, TypeError, ValueError,) as error:

Observed behavior:

WRITER_RETRY attempt 1
WRITER_RETRY attempt 2
WRITER_DEAD_LETTER attempts 3
WRITER_COMMIT_DEAD_LETTER

The failed message was published to: s7801_dead_letter

Results:

Check	Result
Maximum attempts	3
Retry events	2
Dead-letter message produced	Yes
Poison offset committed	Yes
Later valid message processed	Yes
Overall result	PASS

The valid message following the poison message was successfully converted into a draft. Therefore, the poison message did not block later work.

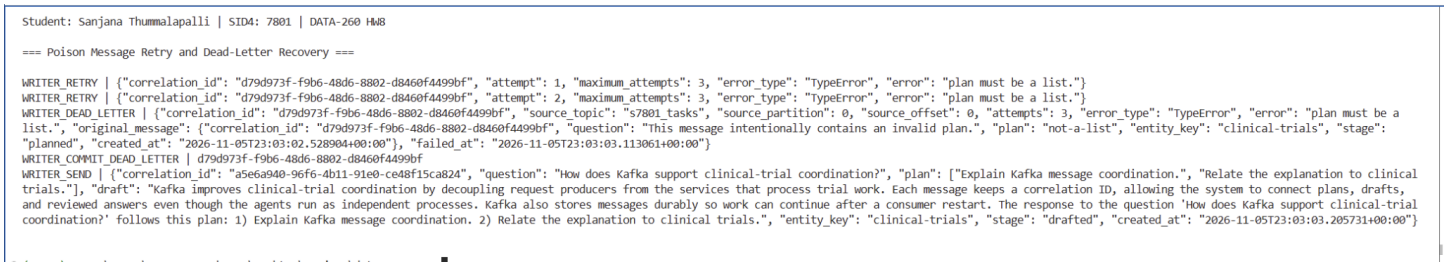


Figure 13: Retry, dead-letter, commit, and later valid-message logs

Ordering with Two Partitions and Two Writers

The ordering experiment used VERIFY_SEED = 267801 and generated 20 related clinical-trial messages.

Before Using an Entity Key

Messages were distributed between partitions 0 and 1 and processed by two Writers.

Observed completion order:

1, 3, 5, 7, 0, 9, 11, 13, 15, 2,
17, 19, 4, 6, 8, 10, 12, 14, 16, 18

Results:

Metric	Before
Messages	20
Partitions	0 and 1
Writer processes	2
Ordering preserved	No

After Using the Related Entity as the Key

The common entity key was: clinical-trial-267801

All related messages entered partition 0 and were completed in order:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9,
10, 11, 12, 13, 14, 15, 16, 17, 18, 19

Results:

Metric	After
Messages	20
Partitions used	1
Related-message order	Preserved

Overall result	PASS

```
Student: Sanjana Thummalapalli | SID4: 7801 | DATA-260 HW8

=== Ordering with Two Partitions and Two Writers ===
DATA-260 Homework 8 Part 3
Ordering and Partitioning Test
Messages per scenario: 20

=== BEFORE: NO ENTITY KEY ===
Completion order: [1, 3, 5, 7, 0, 9, 11, 13, 15, 2, 17, 19, 4, 6, 8, 10, 12, 14, 16, 18]
Partitions: [0, 1]
Writers: ['writer-17656', 'writer-61636']
Ordering preserved: False

=== AFTER: RELATED ENTITY KEY ===
Completion order: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
Partitions: [0]
Writers: ['writer-61636']
Ordering preserved: True

Before used two partitions: True
Before used two writers: True
Before showed disorder: True
After used one partition: True
After preserved order: True
Overall: PASS
JSON: C:\Users\ADS-Research-001\Desktop\sanjana\data260-7801\reports\hw08\raw\ordering_results.json
CSV: C:\Users\ADS-Research-001\Desktop\sanjana\data260-7801\reports\hw08\raw\ordering_results.csv
```

Figure 14: Ordering results showing before disorder and after order 0 through 19

Failure Comparison Table

Failure scenario	Before fix	After fix
Duplicate delivery	A replay created a second UUID and duplicate entity.	Correlation-ID caching returned the original response and prevented a second side effect.
Consumer killed mid-work	A Writer could terminate before completing the task.	The uncommitted task was redelivered after restart and committed only after success.
Poison message	An invalid message could terminate the Writer and block later work.	The Writer used bounded retries, sent the failed message to s7801_dead_letter, committed it, and continued.
Two partitions and two consumers	Related unkeyed messages used two partitions and completed out of order.	The clinical-trial entity key placed related messages in one partition and preserved order.

Benefits Gained

Independent scaling: The Planner, Writer, and Reviewer can scale independently. Multiple Writers can share the two task partitions without creating additional Planner or Reviewer instances.

Decoupled failure and durable work: A Writer failure does not require the Planner to recreate the task. Kafka retains the uncommitted message and redelivers it after recovery.

New Responsibilities

Duplicate handling and idempotency: Kafka may redeliver a message when processing succeeds but the offset is not committed. The application must use correlation IDs and idempotency logic to prevent duplicate side effects.

Ordering, retries, and offset management: Ordering is guaranteed only within one partition. The application must select meaningful message keys, commit offsets after success, bound retries, and route unrecoverable messages to a dead-letter topic.

JMeter Load Test
Configuration

The FastAPI-to-Kafka CREATE endpoint was tested with Apache JMeter.

Setting	Value
Endpoint	POST /hw8/trials
Users	100
Ramp-up	10 seconds
Loop count	1
Total requests	100

Each request used a unique JMeter-generated correlation ID:

```
{
  "operation": "CREATE_CLINICAL_TRIAL",
  "correlation_id": "${__UUID()}",
  "data": {
    "trial_title": "JMeter Kafka Clinical Trial ${__threadNum}",
    "trial_description": "This clinical trial was created during the DATA-260 Homework 8 JMeter load test.",
    "trial_phase": "Phase II",
    "sponsor_name": "San Jose State University",
    "enrollment_target": 150
  }
}
```

Results

Metric	Result
Samples	100
Average response time	72 ms
Minimum response time	63 ms
Maximum response time	169 ms
Error rate	0.00%
Throughput	10.0 requests per second

Duration	10 seconds
Result	PASS

All 100 requests completed successfully. No HTTP or Kafka request errors were observed.

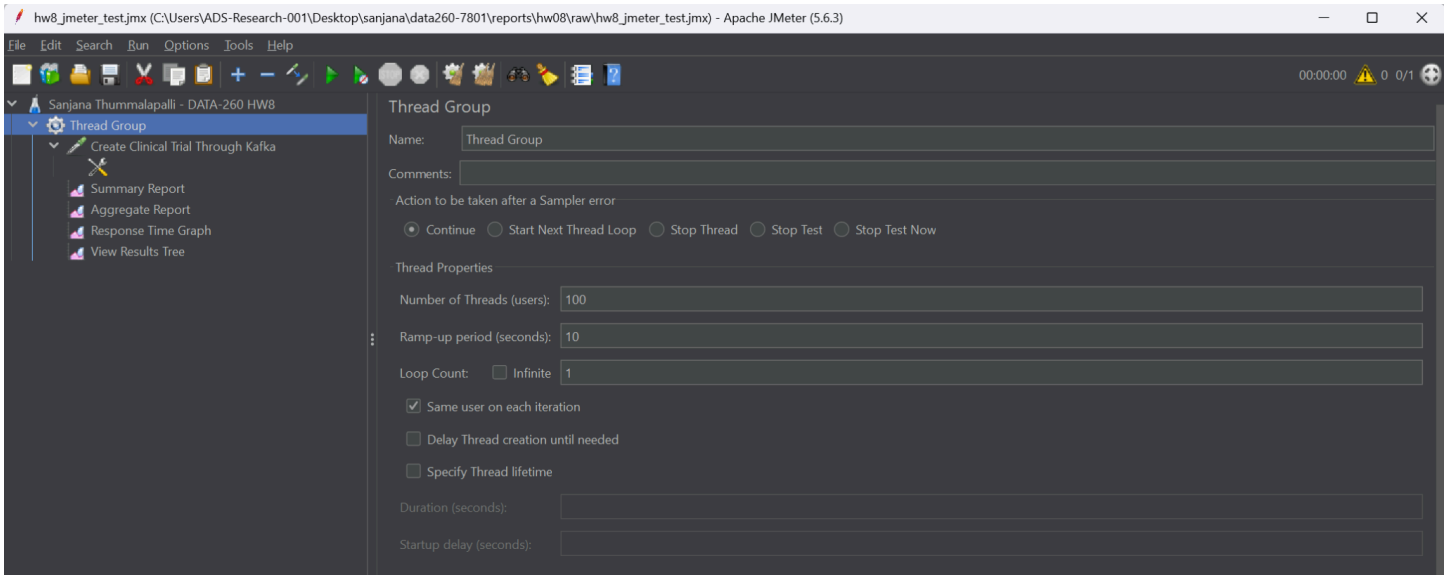


Figure 15 : JMeter Thread Group showing 100 users, 10-second ramp-up, and loop count 1

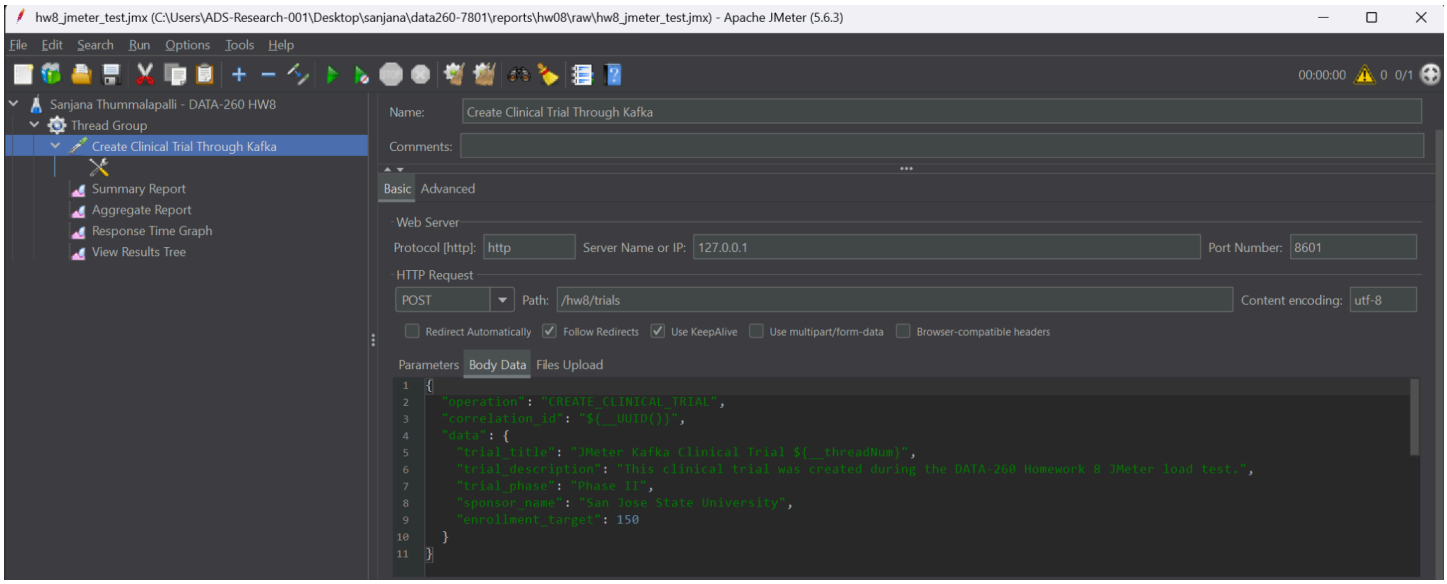


Figure 16 : JMeter HTTP Request and JSON body

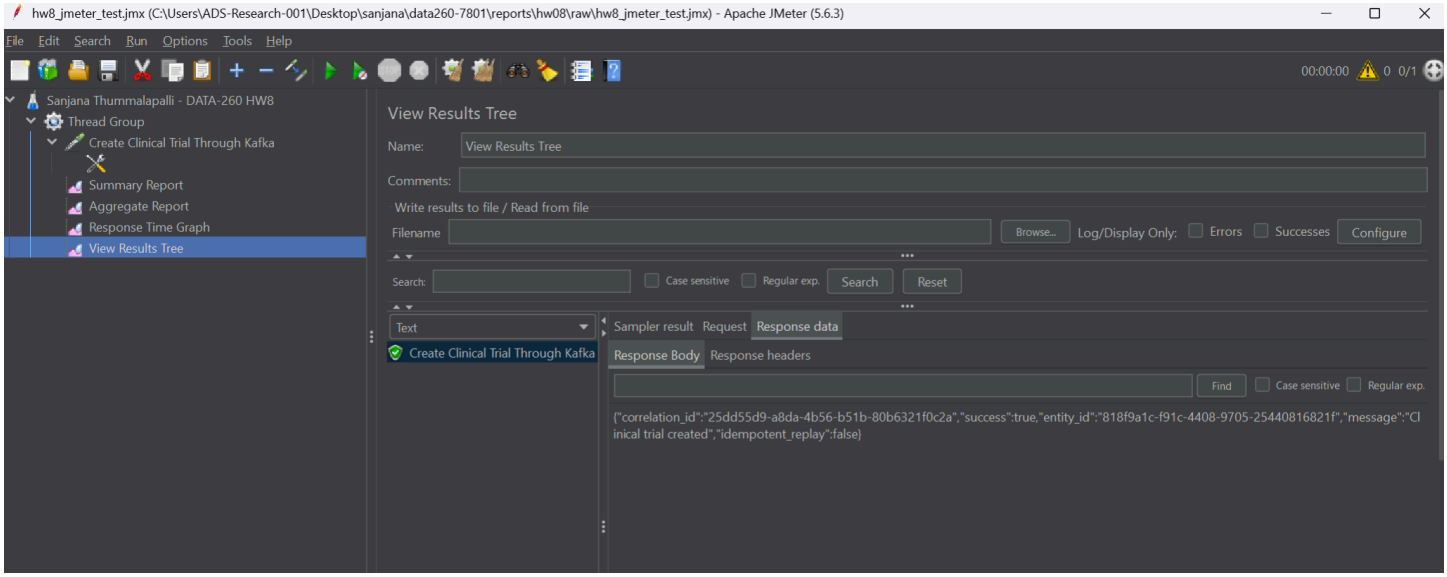


Figure 17 : Successful JMeter response showing success, correlation ID, and entity ID

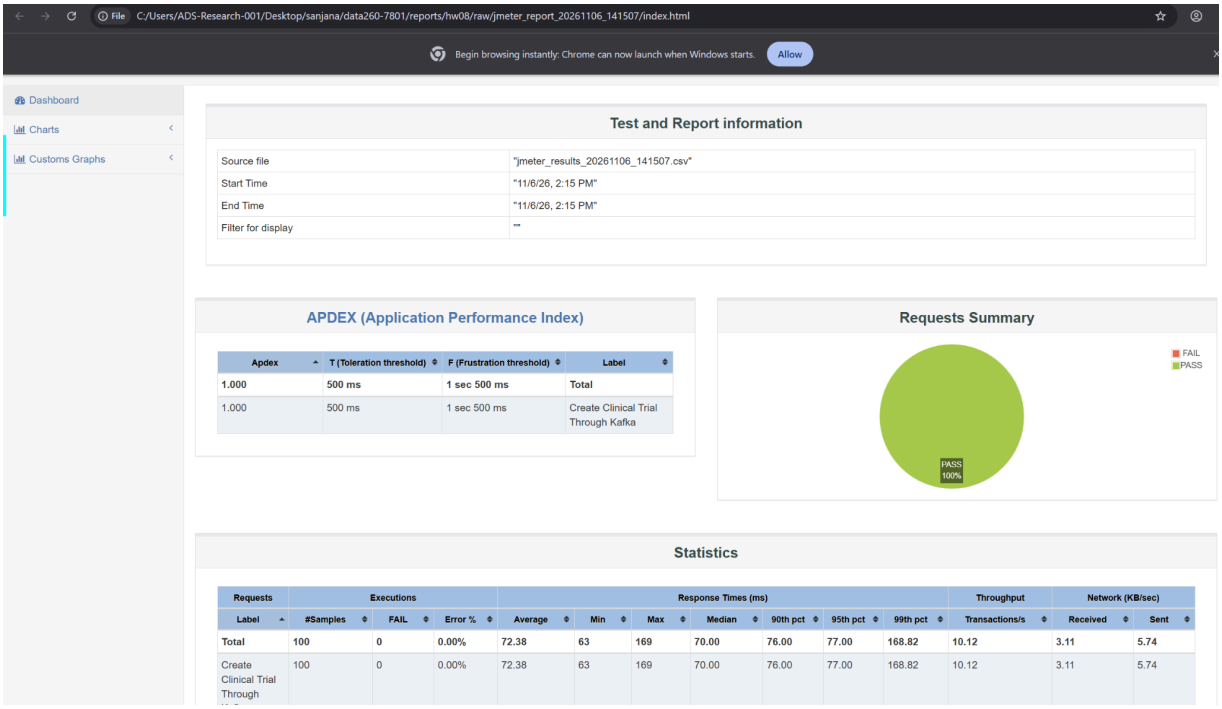


Figure 18 : JMeter HTML dashboard and statistics

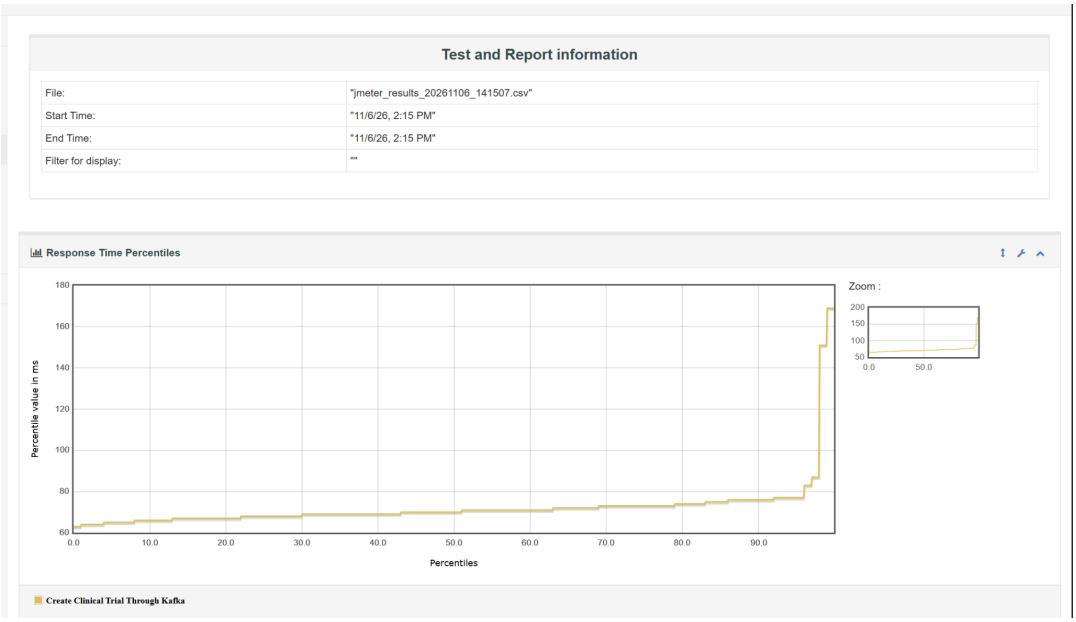


Figure 19: JMeter response-time graph

Automated Verification

The self-check script verified:

- Personal configuration
- Required Kafka topics
- Two task partitions
- CREATE-to-GET round trip
- Correlation-ID preservation
- Duplicate delivery before the fix
- Idempotency after the fix
- Poison-message recovery
- Ordering experiment
- Three-agent pipeline
- Consumer-death redelivery

Final result:

Passed: 11/11
Overall: PASS

The machine-readable output is stored in: reports/hw08/verification.json

```
Student: Sanjana Thummalapalli | SID4: 7801 | DATA-260 HW8

=== Homework 8 Automated Verification ===
DATA-260 Homework 8 Verification
PASS | personal_configuration | SID4=7801, PREFIX=s7801, SEED=7801, VERIFY_SEED=267801
PASS | required_kafka_topics | All required topics exist.
PASS | tasks_topic_two_partitions | s7801_tasks has 2 partitions.
PASS | create_get_round_trip | CREATE followed by GET passed.
PASS | correlation_id_preserved | Correlation ID was preserved.
PASS | duplicate_before_demonstrated | duplicate_before.json: duplicate_side_effect=True
PASS | duplicate_after_idempotent | duplicate_after.json: idempotency_passed=True
PASS | poison_message_recovery | poison_message_results.json: overall_passed=True
PASS | ordering_experiment | ordering_results.json: overall_passed=True
PASS | three_agent_pipeline | Final status=approved
PASS | consumer_death_redelivery | Redelivered and completed IDs: dff8be9d-a81a-4ec8-bdbe-c9ff13dbb3d5

Passed: 11/11
Overall: PASS
Saved: C:\Users\ADS-Research-001\Desktop\sanjana\data260-7801\reports\hw08\verification.json
```

Figure 20 : Automated verification showing 11/11 and Overall PASS

AI Use Statement : What I Used AI For

I used an AI assistant to help interpret the Homework 8 requirements, plan the Kafka architecture, draft portions of the Python code, troubleshoot errors, and prepare verification commands and report structure.

I personally configured Docker Desktop, created and ran the Kafka containers, executed the experiments, inspected logs, terminated and restarted the Writer, ran JMeter, verified JSON evidence, corrected files in VS Code, and confirmed results using py_compile, Ruff, Kafka CLI commands, and runtime output.

AI Output That Was Wrong or Unsuitable

Some initially generated file-creation instructions included PowerShell here-string markers such as @' and '@ | Set-Content. When those wrapper lines were accidentally included inside Python and YAML files, they caused syntax and parsing errors.

The initial Kafka request/response client also occasionally timed out even though the worker had produced a response.

How I Detected the Problems

I detected file-content problems using:

- python -m py_compile
- python -m ruff check
- Docker Compose parsing
- Inspection of the first and last lines of affected files

I identified the Kafka timing issue by comparing client tracebacks with worker logs and matching correlation IDs.

Conclusion

The Homework 8 implementation successfully moved clinical-trial operations and agent coordination from direct in-process calls to Kafka-based asynchronous communication. The system passed the CREATE-to-GET request/response demonstration, three-agent pipeline, duplicate-delivery test, consumer-death recovery, poison-message test, and ordering experiment.

The final JMeter run completed 100 requests with a 0.00% error rate and an average response time of 72 milliseconds. Automated verification passed all 11 checks.